OPEN AGENT HACKATHON 2026 · TRACK 1 — SOLVING FRAGMENTED INTELLIGENCE

RecallWhy

Agents that tell a retailer which recalled products are on its shelves, which lots, which stores, how many customers to call — **and why they believe it.**

Zetaris federated SQL · Meterless H-MEM + World Model · Meterless Nightshift · NVIDIA Nemotron 3 Ultra · built in Cursor

Mike Kim · solo

github.com/mikey92/recallwhy

517 federal recall notices since July. Each one is a manual cross-check.

A regional retailer's **product-safety / QA manager** reads every CPSC and FDA notice and must answer:

- Do we sell this — or something that only looks like it?
- Which lots? Which stores still hold them?
- How many customers bought it since the recall began?
- Who supplied it?

Dec 2025: FDA warning letters to Kroger, Albertsons, Target and Walmart — recalled infant formula still on shelves in 175+ locations across 36 states, up to three weeks. Staff cited "confusion regarding which specific lots were affected."

184 + 333

CPSC recalls and FDA food enforcement reports, 1 Jul – 21 Oct 2026, read live by RecallWhy

The answers live in four systems that were never meant to meet — two **recall feeds**, the **ERP** and the **supplier portal** — each writing a product its own way: UPC-12 in the ERP, GTIN-14 from a third of suppliers, digits buried in notice text. A wrong "not ours" leaves a contaminated lot on the shelf; a wrong "ours" pulls good stock.

2 · SOLUTION

Five agents map the fragments, then investigate every recall and return a verdict with its evidence.

Cartographer

Lists every Zetaris source and column, profiles identifiers, tests cross-source joins. Finds the GTIN-14 suppliers upfront. When the Auditor hits rows the key can't reach, it diagnoses that supplier, tests the fix on all its ship notices, and corrects the map.

Scout

Reads both live feeds and the item master through Zetaris; one federated UPC sweep; opens a case per recall that touches the catalog.

Investigator

UPC / model rules, Nemotron judge for text matches, lot codes, prior decisions from H-MEM, exposure by store and lot with federated SQL.

Auditor

Can block. Re-checks the live notice, quotes and lots verbatim, re-derives every number with its own SQL, checks each brief sentence. ≤2 revisions, then a human.

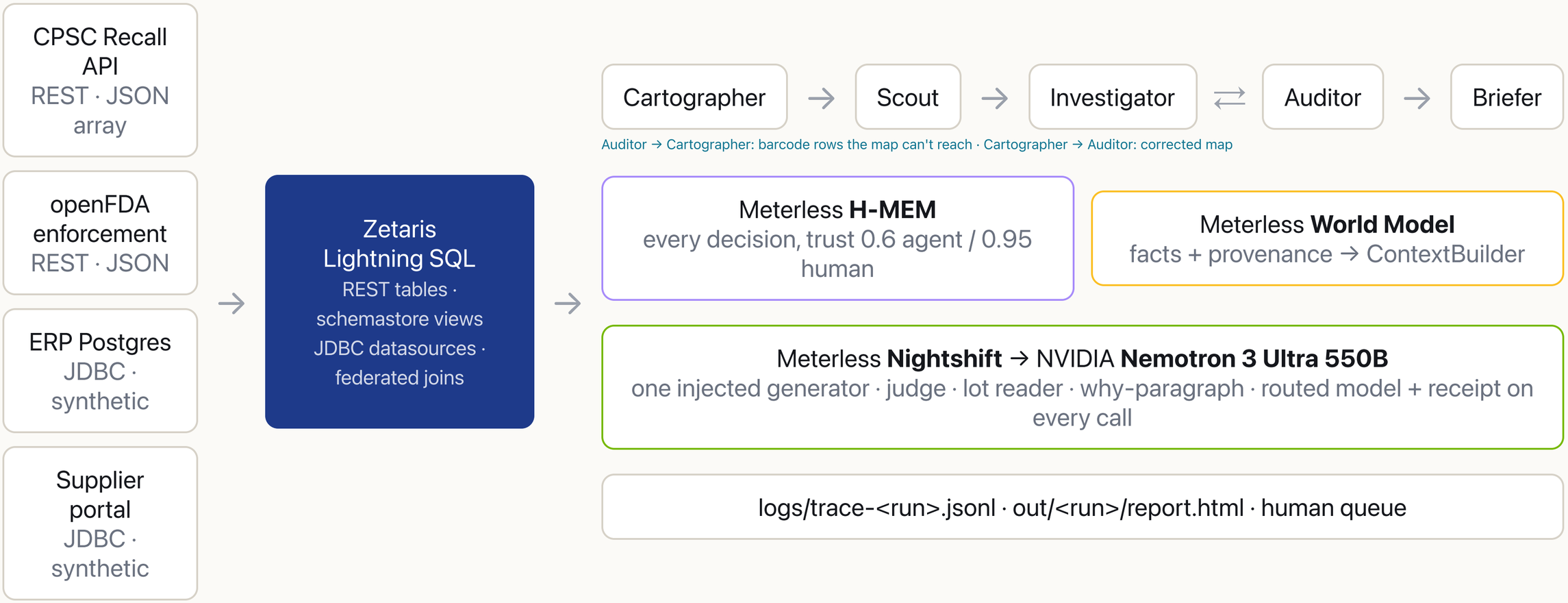
Briefer

Writes only from the World Model context: numbered evidence, each with the SQL or model call behind it.

ACT NOW | NOTIFY | NO EXPOSURE | NOT OURS | NEEDS HUMAN

3 · ARCHITECTURE

Every agent reads data only through Zetaris.



4 · STACK

Small, typed, runnable from a clean clone.

LAYER	WHAT
Data access	Zetaris Lightning SQL over its REST API (`/api/proxy/sql-editor/sqls/run-query`)
Agents	TypeScript on Node 22, no agent framework — five agents, one orchestrator, one trace
Memory & state	Meterless H-MEM and World Model reference engines (Apache-2.0), mounted in `src/engines/`
Model	Meterless **Nightshift** Runtime API (OpenAI-compatible) → NVIDIA **Nemotron 3 Ultra 550B-A55B**; no model and a human queue when Nightshift is absent
Retailer data	Postgres 16 in Docker — synthetic ERP + supplier portal (seeded generator)
Entry points	`POST /run` API · Dockerfile · CLI · live trace viewer
No-credential mode	`SAMPLE_MODE=true` : same agents and SQL on an in-process DuckDB stand-in, labelled SAMPLE
Development	Cursor (agent scaffolding, generator, SQL layer, tests) · 58 tests incl. Meterless engine tests

5 · HOW ZETARIS AND METERLESS ARE USED

Load-bearing, not bolted on.

Zetaris

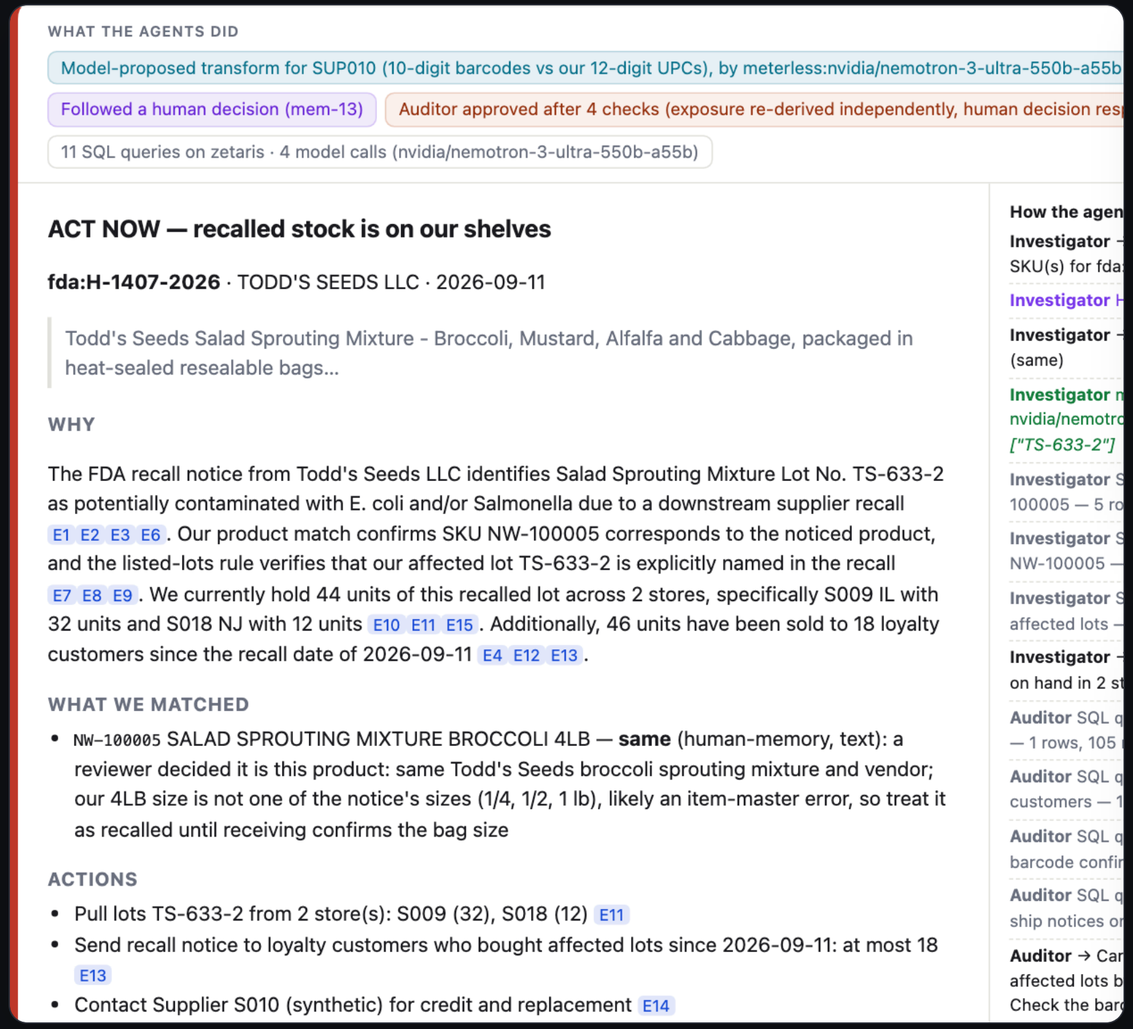
- Cartographer maps sources with Zetaris's catalog: `SHOW DATASOURCES`, `DESCRIBE DATASOURCE TABLE`, `SHOW LIGHTNING DATABASES` → profiles and join tests
- 2 live Lightning REST tables → schemastore views
- 2 JDBC datasources (ERP, supplier portal)
- Scout: **REST × JDBC** join — live FDA notice digits ⋈ ERP UPCs
- Investigator: ship notices ⋈ suppliers ⋈ ERP item master
- Auditor: re-queries the live notice and re-aggregates every number itself
- Setup is code: `npm run setup:zetaris`

Meterless

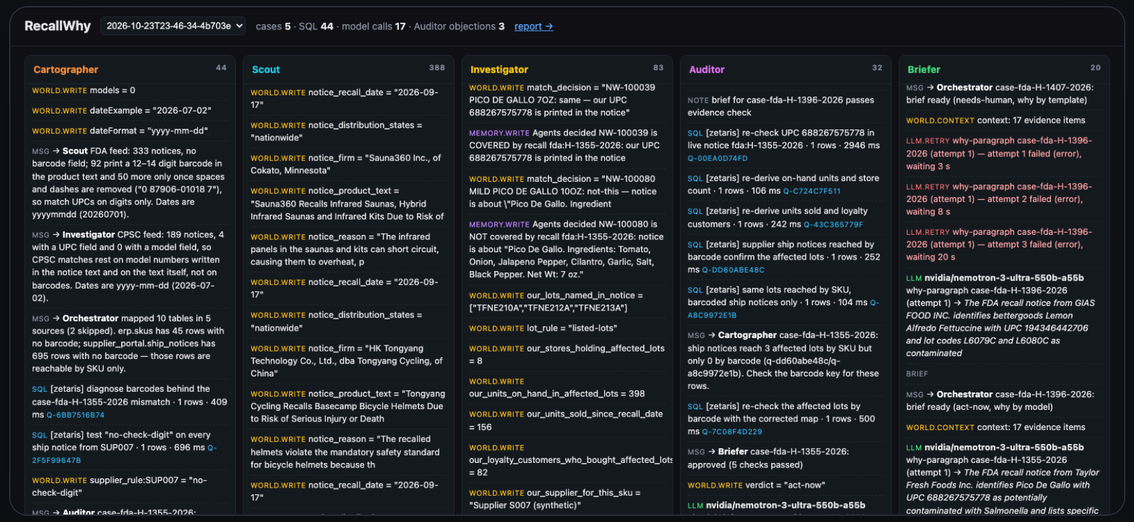
- **H-MEM**: every adjudication stored with trust; human decisions (0.95) are followed on the next run; audited agent decisions are reused without a model call; trust ledger logs reads/writes
- **World Model**: recalls, cases, findings as facts; every write carries provenance (agent, SQL id, model call)
- **ContextBuilder** is the Briefer's only input — it cannot cite what the team didn't establish
- **Nightshift** is the primary AI provider: every model call, with its receipt and routed model in the trace

Map the fragments first. Then trust no one — including the model.

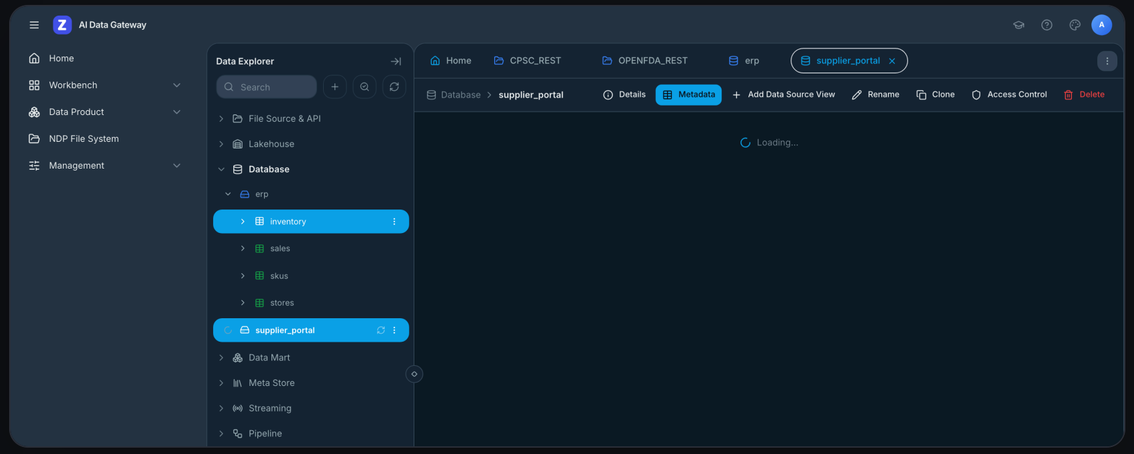
- **The map is a hypothesis the Auditor can send back.** Rows the key can't reach go to the Cartographer, which learns the supplier's rule (padding, dropped check digit) and proves it on all that supplier's ship notices.
- **The model proposes, SQL proves.** For a format the code has no rule for (a 10-digit core), Nemotron 3 Ultra proposes a transform from sampled pairs in a fixed op set; it is adopted only if it holds on every one of the supplier's rows (269 of 269). Judge off: **22** of 194 caught with known rules, **103** with proposals, **11** without the Cartographer.
- **Re-derives every number in Zetaris** with differently written SQL; a mismatch blocks the brief.
- **Re-reads the live notice** (REST table) to confirm a UPC match, instead of trusting the parser.
- **"Cheese ≠ pepperoni"**: a quote can be copied from the notice and still describe another variety — every product word of our item must appear in the notice. Live run: blocked, re-judged, ruled *not ours*.
- **Lot codes the model reads must appear verbatim** in `code_info`; invented lots are dropped.
- **Brief sentences are checked** against the evidence they cite; a draft that fails the check is rewritten or replaced by the template.
- **Human decisions change the next run** through H-MEM, and the brief cites them.



Each case opens with what the agents did (here: a transform Nemotron proposed, proven by SQL on 299/299), then the cited brief and the trace.



Live viewer: the agents' events as they happen, from the trace file.



Zetaris: REST feeds and JDBC sources registered by `setup:zetaris`.

8 · RESULTS

187 of 194 recalled items caught. The Auditor cuts missed recalls 10 → 3.

SETUP (440 RECALL/SKU PAIRS)	CAUGHT (OF 194)	WRONGLY DISMISSED	FALSE ALARMS (OF 246)	TO HUMAN
No Cartographer — rules only	11	0	0	429
Cartographer, known rules only	22	0	0	418
Cartographer + model-proposed transforms (no judge)	103	0	0	337
+ Nemotron 3.5 Lightning 30B via Nightshift	139	10	2	66
+ NVIDIA Nemotron 3 Ultra via Nightshift	**187**	**3**	**6**	**23**

TEXT MATCHES (90 COVERED OF 336)	CAUGHT	WRONGLY DISMISSED	FALSE ALARMS	TO HUMAN
Ultra: judge's first answer	80	10	10	0
Ultra: after the Auditor loop	**85**	**3**	**6**	**21**

417 of 440 pairs (95%) settled without a person, 408 correctly; 23 go to a reviewer with the reason. The Auditor checks dismissals too. 0 of 538 model calls failed.

Real Investigator ⇄ Auditor loop on every candidate pair from 517 recalls (sample-mode data). Covered = SKU generated from that recall, or its UPC printed in the notice. Retailer data is synthetic. Details: eval/EVAL.md.

9 · NEXT

From demo to a safety desk.

- More feeds through Zetaris: USDA FSIS, NHTSA equipment (car seats), Health Canada
- Lot capture from receiving scans instead of the item master
- Customer notification through the loyalty system — counts today, messages tomorrow
- Cartographer onboarding new feeds on its own: catalog → profile → join test
- Markovian engine for daily long-running sweeps
- Review screen for a team: assignment, and each decision's history from the H-MEM trust ledger

Limits today: synthetic retailer data; proposed transforms are limited to string ops; lexical Auditor checks; CPSC rarely prints UPCs; hosted model rate limits make a full evaluation take over an hour.